University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 4	**CLOs to be covered:** CLO1
Lab Title: Nested Loops	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 4

Lab Goals / Objectives:

• Understand the concept of nested loops (loops within loops)

• Learn different types of function arguments: positional, keyword, default

• Understand variable-length arguments (*args)

• Apply nested loops to create geometric patterns

• Master function definition with multiple parameter types

• Learn to return multiple values and use them effectively

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. Nested Loops

A nested loop is a loop inside the body of another loop. The inner loop executes completely for each iteration of the outer loop. Both for loops and while loops can be nested.

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

In this example, the inner loop runs 3 times for each of the 3 iterations of the outer loop, resulting in 9 total iterations.

2. Using Nested Loops for Patterns

Nested loops are commonly used to generate geometric patterns. The outer loop typically controls the rows, and the inner loop controls the columns or elements within each row.

3. Positional Arguments

Positional arguments are passed to a function in the order they are defined. The position of the argument determines which parameter it corresponds to:

```
def greet(name, age):
    print(name, age)
greet('Ali', 20)  # name='Ali', age=20
```

4. Keyword Arguments

Keyword arguments are passed by explicitly specifying the parameter name. This allows arguments to be passed in any order:

```
def greet(name, age):
    print(name, age)
greet(age=20, name='Ali')
```

5. Default Arguments

Default arguments have a default value specified in the function definition. If the caller does not provide a value, the default is used:

```
def power(base, exponent=2):
    return base ** exponent
print(power(5))      # 25 (5^2)
print(power(2, 3))   # 8 (2^3)
```

6. Variable-Length Arguments (*args)

The *args syntax allows a function to accept any number of positional arguments. These arguments are collected into a tuple:

```
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3, 4))  # 10
```

7. The f-string Formatting

f-strings provide an elegant way to embed expressions inside string literals for formatted output:

```
name = 'Ali'
age = 20
print(f'Name: {name}, Age: {age}')
```

8. The sum() and len() Built-in Functions

• sum(iterable) - Returns the sum of all items in an iterable

• len(object) - Returns the number of items in an object

```
numbers = [1, 2, 3, 4]
print(sum(numbers))  # 10
print(len(numbers))  # 4
```

9. Arithmetic Operators

• ** (Exponent): Raises the left operand to the power of the right

• % (Modulus): Returns the remainder of division

• // (Floor Division): Returns the integer part of division

10. Comparison Operators

• > (Greater than), < (Less than)

• >= (Greater than or equal), <= (Less than or equal)

• == (Equal to), != (Not equal to)

• and (Logical AND), or (Logical OR)

Lab Work:

Task 1: Nested Multiplication Table (While)

Write a program that uses nested while loops to print a multiplication table for numbers 1 to 3, each up to 10.

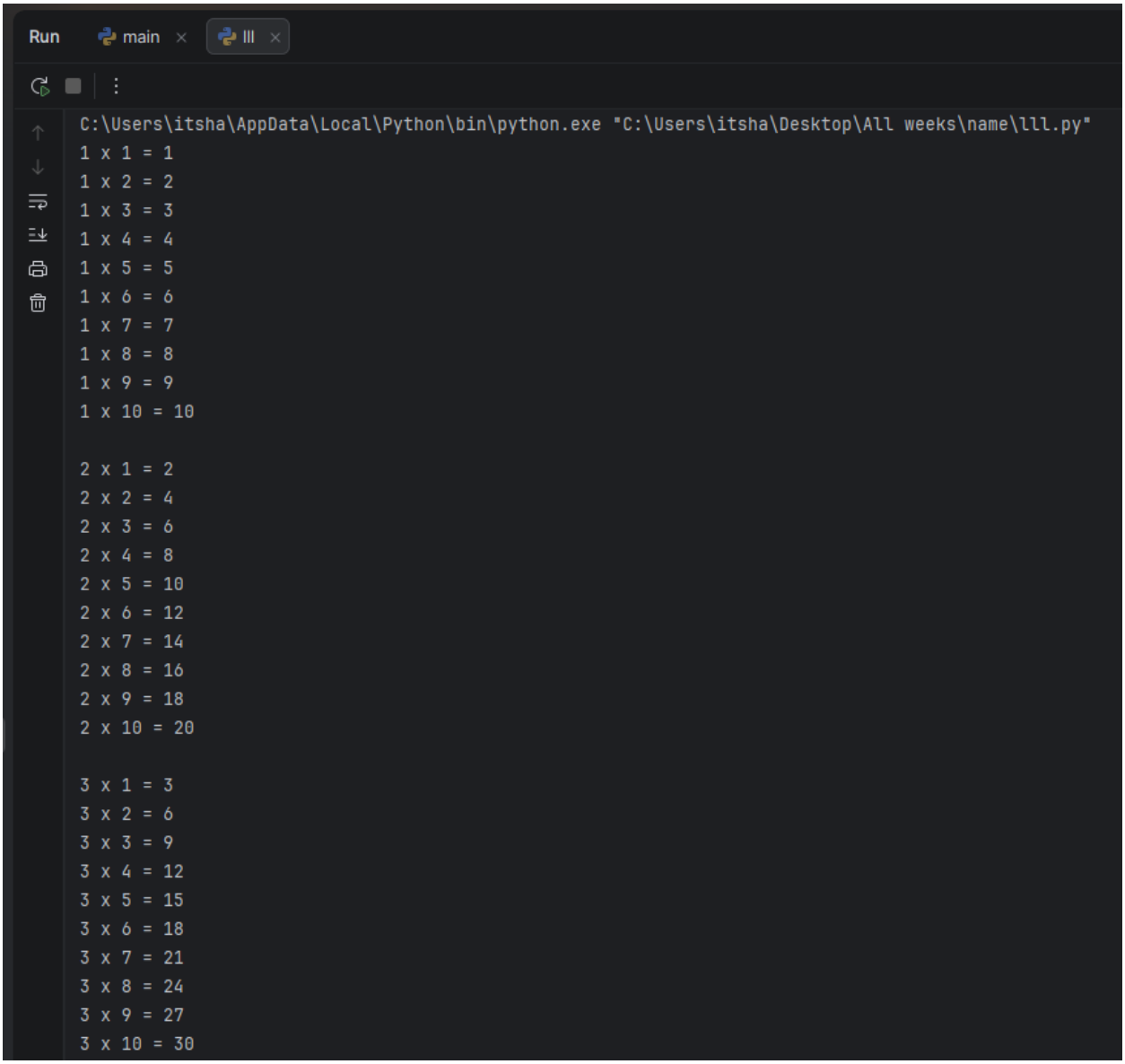
Code Summary:

The outer while loop controls the row number (1 to 3) and the inner while loop controls the column number (1 to 10), printing each row's multiplication results before moving to the next row.

Key Code Lines:

```
# Task 1: Nested Multiplication Table
i = 1
while i <= 3:
    j = 1
    while j <= 10:
        print(i, "x", j, "=", i * j)
        j += 1
    print()
    i += 1
```

Output:



Task 2: Decreasing Number Triangle (While)

Write a program that uses nested while loops to print a decreasing triangle of numbers.

Code Summary:

The outer while loop counts down the number of rows, while the inner while loop prints numbers from 1 up to the current row count, forming a shrinking triangle.

Key Code Lines:

```
# Task 2: Decreasing Number Triangle
i = 5
while i >= 1:
    j = 1
    while j <= i:
        print(j, end=" ")
        j += 1
```

```
    print()
    i -= 1
```

Output:

Task 3: Count Vowels and Consonants (Nested For)

Write a program that counts the vowels and consonants in a sentence using a nested for loop.

Code Summary:

The outer for loop scans each character of the sentence, and the inner for loop checks that character against every vowel; a for-else construct increments the consonant count if no vowel match is found.

Key Code Lines:

```
# Task 3: Count Vowels and Consonants
sentence = input("Enter a sentence: ")

vowels = 0
consonants = 0

for ch in sentence:
    if ('a' <= ch <= 'z') or ('A' <= ch <= 'Z'):
        for v in "aeiouAEIOU":
            if ch == v:
                vowels += 1
                break
        else:
            consonants += 1

print("Number of vowels:", vowels)
print("Number of consonants:", consonants)
```

Output:

Task 4: Increasing and Decreasing Star Pyramid

Write a program that uses nested for loops to print an increasing star pattern followed by a decreasing star pattern.

Code Summary:

Two separate nested for-loop blocks are used: the first builds rows of stars that grow in size, and the second builds rows that shrink, together forming a simple pyramid effect.

Key Code Lines:

```
# Task 4: Increasing and Decreasing Star Pattern
for i in range(1, 5):
    for j in range(i):
        print("*", end=" ")
    print()

for i in range(3, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

Output:

Conclusions:

- Successfully understood and applied nested loops for pattern generation
- Mastered different types of function arguments: positional, keyword, and default
- Learned to use variable-length arguments (*args) for flexible function calls
- Applied conditional statements within functions for decision-making
- Understood the concept of returning values from functions
- Developed multiple utility functions demonstrating various function concepts
- Gained practical experience in modular and reusable code design